Python Full Handbook

1. Introduction to Python

★ What is Python?

Python is a high-level, interpreted, general-purpose programming language. It was created by Guido van Rossum and released in 1991. Python is designed to be easy to read and simple to use.

Key Features

- ✓ Easy to learn and use
- ✓ Readable and clean syntax
- ✓ Interpreted Language
- ✓ High-level Language
- ✓ Platform Independent
- ✓ Object-Oriented
- ✓ Large Standard Library
- ✓ Open Source

★ Why Python?

- Simple and easy to learn.
- Wide range of applications.
- Large community support.
- Cross-platform compatibility.
- Regular updates and improvements.

★ Python Versions

Python 2.x	Python 3.x
• Old version	• Latest version
• Syntax is complex	• Syntax is simple
• Print is a statement (print "Hello")	• Print is a function (print("Hello")
• No longer supported	• Recommended to use

Note: We will use Python 3.x in this handbook.

★ Applications of Python

- Web Development (Django, Flask)
- Data Science & Machine Learning
- Artificial Intelligence
- Automation / Scripting
- Game Development
- Desktop GUI Applications
- Cyber Security, Networking and more...

★ First Python Program

```
# This is a simple Python program
print("Hello, World!")
```

→ Output: Hello, World!

Every Python program starts with indentation and simple syntax.

★ How Python Works?

Write code in Python file (.py) → Python Interpreter reads the code → Interpreter converts it to bytecode → Python Virtual Machine (PVM) executes it → Output is displayed on screen

★ Comments in Python

```
# This is a single line comment
# Comments are used to explain the code
```

← Comments are ignored by the interpreter.

Python Full Handbook

2. Variables and Data Types

★ What is Variable ?

A variable is a name given to a memory location in a program. It is used to store data.

Python has no command for declaring a variable.

Examples

```
x = 10              # integer
name = "Sagar"      # string
price = 25.5        # float
is_active = True    # boolean
```

Note :
No need to specify data type in Python.

★ Python Data Types

Data Type	Description	Example
int	Integer numbers	x = 10
float	Decimal numbers	pi = 3.14
str	String (text)	name = "Python"
bool	Boolean (True / False)	is_active = True
list	Ordered, changeable collection	nums = [1, 2, 3, 4]
tuple	Ordered, unchangeable collection	t = (1, 2, 3)
dict	Key-Value pairs	d = {"a": 1, "b":2}
set	Unordered collection of unique items	s = {1, 2, 3}

Type Checking

```
x = 100
print(type(x))      # <class 'int'>

name = "Hello"
print(type(name))   # <class 'str'>

val = 3.14
print(type(val))    # <class 'float'>
```

★ Type Conversion

We can convert one data type into another.

int()	float()	str()	list()	tuple()	set()
x = "10"	x = "3.14"	x = 25	x = (1,2,3)	x = [1,2,3]	x = [1,2,2,3]
y = int(x)	y = float(x)	y = str(x)	y = list(x)	y = tuple(x)	y = set(x)
print(y) # 10	print(y) # 3.14	print(y) # '25'	print(y) # [1,2,3]	print(y) # (1,2,3]	print(y) # {1,2,3}

★ Input from User

We can take input from user using input() function.

```
name = input("Enter your name: ")      # input is always string
age = int(input("Enter your age: "))   # convert into integer
print("Name: ", name)
print("Age: ", age)
```

Important

- ✓ input() always returns string.
- ✓ Use type conversion when needed.
- ✓ Variables are case-sensitive.

→ Variables are containers for storing data and data types define the type of data. ←

PYTHON FULL HANDBOOK

3. OPERATORS IN PYTHON

★ What is Operator ?

An operator is a symbol that performs an operation on one or more operands (values).

Example : a + b , x * y , a > b

Types of Operators
1. Arithmetic Operators
2. Comparison (Relational) Operators
3. Logical Operators
4. Assignment Operators
5. Identity Operators
6. Membership Operators
7. Bitwise Operators

Note : Operators are used to perform operations.

★ 1. Arithmetic Operators

Operator	Name	Example	Meaning
+	Addition	x + y	Add two values
-	Subtraction	x - y	Subtract y from x
*	Multiplication	x * y	Multiply two values
/	Division	x / y	Divide x by y (float result)
//	Floor Division	x // y	Divide and get integer result
%	Modulus	x % y	Remainder of division
**	Exponent	x ** y	x raised to power y

Example

```
a = 10
b = 3
print(a + b)      # 13
print(a - b)      # 7
print(a * b)      # 30
print(a / b)      # 3.333...
print(a // b)     # 3
print(a % b)      # 1
print(a ** b)     # 1000
```

★ 2. Comparison (Relational) Operators

Operator	Name	Example	Meaning
==	Equal to	a == b	True if a is equal to b
!=	Not equal to	a != b	True if a is not equal to b
>	Greater than	a > b	True if a is greater than b
<	Less than	a < b	True if a is less than b
>=	Greater than or equal to	a >= b	True if a is greater than or equal to b
<=	Less than or equal to	a <= b	True if a is less than or equal to b

Example

```
a = 10
b = 5
print(a == b)     # False
print(a != b)     # True
print(a > b)      # True
print(a < b)      # False
print(a >= b)     # True
print(a <= b)     # False
```

★ 3. Logical Operators

Operator	Name	Example	Meaning
and	Logical AND	x and y	True if both x and y are True
or	Logical OR	x or y	True if any one of x or y is True
not	Logical NOT	not x	True if x is False

Example

```
x = True , y = False
print(x and y)    # False
print(x or y)     # True
print(not x)      # False
```

★ 4. Assignment Operators

Operator	Example	Same As	Operator	Example	Same As
=	x = 5	x = 5	/=	x /= 3	x = x / 3
+=	x += 3	x = x + 3	%=	x %= 3	x = x % 3
-=	x -= 3	x = x - 3	//=	x //= 3	x = x // 3
*=	x *= 3	x = x * 3	**=	x **= 3	x = x ** 3

Example

```
x = 10
x += 5      # x = 15
x *= 2      # x = 30
x -= 10     # x = 20
```

→ Operators are the building blocks of expressions in Python. ←

Note : Use parentheses ( ) to control the order of operations.

Python Full Handbook

4. Control Flow Statements

★ If Statement

The if statement is used to execute a block of code if a condition is True.

Syntax :

```
if condition:
    # block of code
```

Example

```
x = 10
if x > 5:
    print("x is greater than 5")

# Output: x is greater than 5
```

Note :

Indentation is very important in Python. It shows which statements belong to a block.

★ If-Else Statement

The if-else statement is used to execute one block of code if the condition is True and another block if it is False.

Syntax :

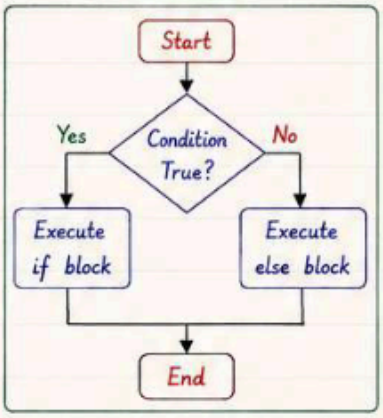
```
if condition:
    # block of code (if True)
else:
    # block of code (if False)
```

Example

```
x = 3
if x % 2 == 0:
    print("x is even")
else:
    print("x is odd")

# Output: x is odd
```

Flowchart



★ If-Elif-Else Ladder

Used when we have multiple conditions to check.

Syntax :

```
if condition1:
    # block of code
elif condition2:
    # block of code
elif condition3:
    # block of code
...
else:
    # block of code
```

Example

```
marks = 85
if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
elif marks >= 50:
    print("Grade: C")
else:
    print("Grade: F")

# Output: Grade: B
```

Number	Output
95	Grade: A
80	Grade: B
60	Grade: C
40	Grade: F

Key Point :

- ✓ The first True condition is executed.
- ✓ Once a block is executed, the rest are skipped.
- ✓ The else block is optional.

★ Nested If Statement

An if statement inside another if statement.

Syntax :

```
if condition1:
    if condition2:
        # block of code
    else:
        # block of code
else:
    # block of code
```

Example

```
x = 15
if x > 0:
    if x % 2 == 0:
        print("Positive Even")
    else:
        print("Positive Odd")
else:
    print("Negative Number")

# Output: Positive Odd
```

Important !

- ✓ Control flow statements help in decision making.
- ✓ They make programs smart and efficient.
- ✓ Practice more examples to get comfortable.

Think → Code → Run → Debug → Improve

PYTHON FULL HANDBOOK

5. TYPE CASTING AND USER INPUT

★ Type Casting

Type casting means converting one data type into another.

Python has two types of casting :

1. Implicit Type Casting (Automatic)
2. Explicit Type Casting (Manual)

Implicit Type Casting (Automatic)

Example	Explanation	Result
x = 10 y = 3.5	int + float → float	13.5
x = 5 y = 2+3j	int + complex → complex	(5+3j)
x = True	bool is converted to int (1)	1

Explicit Type Casting (Manual)

Function	Description	Example
int()	Convert to integer	int(3.14) → 3
float()	Convert to float	float(10) → 10.0
str()	Convert to string	str(25) → "25"
bool()	Convert to boolean	bool(0) → False
complex()	Convert to complex	complex(5) → (5+0j)

★ Type Conversion Examples

```
int(10.9)      # 10
float(7)       # 7.0
str(100)       # '100'
bool(5)        # True
bool(0)        # False
complex(8)     # (8+0j)
```

Note :
Explicit casting is required when Python does not know how to convert automatically.

★ User Input in Python

Python has input() function to take input from user.

Function	Description	Example	Output (if user enters 25)	Data Type
input()	Always returns string	name = input("Enter name: ") print(name)	Enter name: Sagar Sagar	str
int(input())	Converts input to integer	age = int(input("Enter age: ")) print(age)	Enter age: 25 25	int
float(input())	Converts input to float	price = float(input("Enter price: ")) print(price)	Enter price: 12.75 12.75	float

★ Practice Examples

1. Take two numbers from user and print their sum.

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print("Sum =", a + b)
```

2. Take radius from user and find area of circle.

```
r = float(input("Enter radius: "))
area = 3.14 * r * r
print("Area =", area)
```

3. Take your name from user and print length of name.

```
name = input("Enter your name: ")
print("Length =", len(name))
```

Important Points

- ✓ All variables must be declared before use.
- ✓ Python is case-sensitive.
- ✓ Use meaningful variable names.
- ✓ Choose correct data type to save memory.
- ✓ Use type casting when needed.
- ✓ Always take input and handle data carefully.



Learning operators, type casting and input is the base of Python programming.



6. FUNCTIONS IN PYTHON

★ What is a Function?

A function is a block of code that performs a specific task. It helps in code reusability, better organization and reduces repetition.

★ Syntax

```
def function_name(parameters):
    statements
    return value
```

★ Example 1 : No Parameters

```
def greet():
    print("Hello, Python!")

greet()
# Output : Hello, Python!
```

★ Example 2 : With Parameters

```
def add(a, b):
    sum = a + b
    print("Sum =", sum)

add(5, 7)
# Output : Sum = 12
```

★ Example 3 : With Return Value

```
def multiply(x, y):
    result = x * y
    return result

ans = multiply(4, 3)
print("Product =", ans)
# Output : Product = 12
```

★ Types of Arguments

Type	Description	Example
Positional Arguments	Arguments passed in correct position.	`add(5, 7)`
Keyword Arguments	Arguments passed using parameter names.	`add(b = 7, a = 5)`

Default Arguments	Parameter has a default value when no argument is passed.	`def greet(name = "Guest"):`
		`  print("Hello,", name)`
		`greet()  # Hello, Guest`

★ Key Points

- Use functions to make code modular and easy to manage.
- A function can return a value using 'return' statement.
- Parameters are optional but make functions flexible.
- Functions improve readability and reduce repetition.



7. Lambda Functions

★ What is Lambda Function?

A lambda function is a small, anonymous function defined using the 'lambda' keyword.
It can have any number of arguments but only one expression.

Syntax

```
lambda arguments : expression
```

★ Examples

1. No Arguments

```
f = lambda : "Hello"
print(f())   # Hello
```

2. One Argument

```
f = lambda x : x * 2
print(f(5))   # 10
```

3. Two Arguments

```
f = lambda x, y : x + y
print(f(3, 4))   # 7
```

★ Use of Lambda with Built-in Functions

Function	Example
sorted()	people = [("Aman", 22), ("Rohit", 20), ("Neha", 23)] sorted(people, key = lambda x : x[1]) # sort by age
map()	nums = [1, 2, 3, 4] list(map(lambda x : x*x, nums)) # [1, 4, 9, 16]
filter()	nums = [1, 2, 3, 4, 5, 6] list(filter(lambda x : x % 2 == 0, nums)) # [2, 4, 6]

★ Key Points

- Lambda functions are single-expression functions.
- They are mainly used with built-in functions.
- They improve code readability in small operations.
- They are anonymous (no function name).



Example: Square of all numbers using lambda and map()

```
nums = [1, 2, 3, 4]
squares = list(map(lambda x : x ** 2, nums))
print(squares)   # [1, 4, 9, 16]
```

[1, 2, 3, 4]
↓
[1, 4, 9, 16]

8. Return Statement

★ What is return?

The return statement is used inside a function to send a value back to the caller.
It terminates the function and returns the specified value.

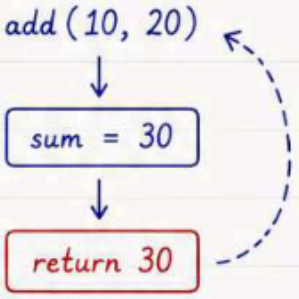
Syntax

```
return expression
```

★ Example 1 : Returning a Value

```
def add(a, b):
    sum = a + b
    return sum

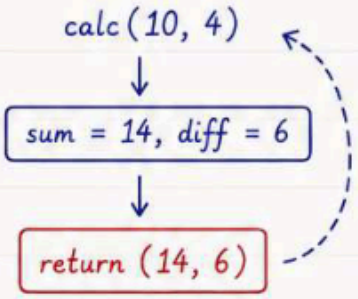
result = add(10, 20)
print(result)      # Output: 30
```



★ Example 2 : Returning Multiple Values

```
def calc(a, b):
    sum = a + b
    diff = a - b
    return sum, diff

s, d = calc(10, 4)
print(s, d)    # Output: 14 6
```

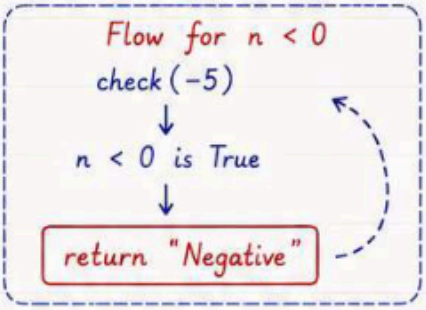


★ Example 3 : Early Return

```
def check(n):
    if n < 0:
        return "Negative"
    else:
        return "Positive"

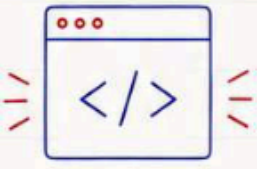
print(check(-5))   # Output: Negative
print(check(7))    # Output: Positive
```

★ Function stops executing as soon as return is hit.



★ Key Points

- return sends a value back from the function.
- A function can return multiple values (as tuple).
- If no return is used, the function returns None by default.
- Code after return is not executed.



9. Types of Functions

★ Built-in Functions

Python has many built-in functions that are always available to use.

Some commonly used built-in functions:

- print()
- input()
- str()
- len()
- int()
- max()
- type()
- float()
- min()

Examples

```
print("Hello")      # Hello
len([1, 2, 3])      # 3
type(10.5)          # <class 'float'>
max(4, 7, 2)        # 7
min('a', 'z')       # 'a'
int("25")           # 25
float("3.14")       # 3.14
```

★ User-defined Functions

Functions created by the user using 'def' keyword are called user-defined functions.

Syntax

```
def function_name(parameters):
    statements
    return value
```

Example

```
def square(n):
    return n * n
result = square(5)
print(result)    # 25
```

★ Difference Between Built-in and User-defined Functions

Feature	Built-in Functions	User-defined Functions
Defined by	Python	User
Availability	Always available	Need to be created
Examples	print(), len(), max()	square(), add(), calc()
Flexibility	Fixed	More flexible

★ Key Points

- Functions help in organizing code into reusable blocks.
- Built-in functions save time and effort.
- User-defined functions give more control and flexibility.
- Functions make programs shorter, cleaner and easy to maintain.

